

SmartSpend AI

Personal Finance & Spending Intelligence — Project Report

A browser-based personal-finance analytics application: upload a transaction file (or try the bundled demo dataset) and instantly get spending analysis, unusual-transaction detection, a forecast, budget comparison, and a financial health score — with a downloadable Excel report. Built as a static website (HTML / CSS / JavaScript) with no backend server; every calculation runs locally in the browser.

Live repository	github.com/Sadhiya0201/SmartSpendAI
Primary stack	HTML5, CSS3, vanilla JavaScript (ES6), Chart.js, SheetJS
Deployment target	Static hosting (Vercel / GitHub Pages)
Report generated	16 August 2026

Contents

01	Project Overview
02	Objectives
03	Technology Stack
04	How It Was Built — Architecture & Workflow
05	Data Science Methods Used
06	Input — What the User Provides
07	Output — Application Walkthrough (with screenshots)
08	Sample Results on the Demo Dataset
09	Testing & Verification
10	Deployment
11	Limitations & Future Improvements
12	Conclusion

01 Project Overview

SmartSpend AI turns a raw transaction export (CSV or Excel) into an interactive personal-finance analysis. The person uploads their own file — or clicks a demo button to explore a realistic pre-built dataset — and the application automatically cleans the data, engineers analytical features, and presents spending patterns, unusual transactions, a short-term forecast, a budget comparison, and a transparent financial health score, entirely inside the browser tab.

The project intentionally demonstrates three skill areas together rather than in isolation: **data cleaning and analysis** (the engineering behind every number shown), **applied data-science techniques** (anomaly detection, forecasting, a weighted scoring model), and **front-end engineering** (a deliberately designed interface, not a default template). No AI/LLM APIs are used anywhere — every insight is produced by explicit, auditable calculations.

Key facts

Aspect	Detail
Type	Fully client-side static website — no backend server or database
Demo dataset	579 transactions across 12 months (Aug 2025 – Jul 2026), 10 spending categories
Core files	index.html, style.css, engine.js, app.js, report.js, demo_data.js
External libraries	Chart.js (charts), SheetJS/xlsx (Excel import & export)
Everything else	Parsing, cleaning, feature engineering, analysis, scoring — all hand-written JS

02 Objectives

- Accept **any** reasonably-formatted transaction file, not one fixed schema — via automatic column detection with a manual mapping fallback when detection is ambiguous.
- Apply genuine, explainable data-science techniques for anomaly detection and forecasting rather than a black-box or LLM-generated result.
- Never fabricate a number. Every figure on screen is calculated directly from the loaded data; any feature the data can't support (e.g. no income column, only one month of history) is explicitly labeled *Not Applicable* rather than estimated.
- Ship both an interactive experience and a downloadable, formatted Excel report from the same underlying calculations.
- Run entirely client-side — nothing the user uploads is ever sent to a server.

03 Technology Stack

Layer	Technology	Why
Structure	HTML5	Single-page application shell
Styling	CSS3 (custom, no framework)	Full control over a distinctive visual identity
Logic / analysis	Vanilla JavaScript (ES6)	Zero build step; runs anywhere a browser does
Charts	Chart.js (CDN)	Lightweight, well-documented charting
Spreadsheet I/O	SheetJS / xlsx.js (CDN)	Read uploaded .xlsx files and write the Excel report
Typography	Fraunces, Inter, IBM Plex Mono (Google Fonts)	Serif headings + monospaced figures for a ledger feel
Hosting	Vercel / GitHub Pages	Zero-config static hosting, auto-deploy on git push

Deliberately excluded: LLM/AI APIs, a backend server, and a database. All "intelligence" in the app comes from statistics implemented directly in JavaScript (percentiles, IQR, Z-scores, linear regression, weighted scoring) — not a call to an external model.

04 How It Was Built — Architecture & Workflow

The codebase is split so that **all calculation logic is independent of the browser DOM** — the analysis engine could run under Node.js, in a browser, or in a test harness unchanged. This separation is what made rigorous testing possible before ever touching a UI.

File responsibilities

File	Responsibility
engine.js	Pure analysis engine: CSV parsing, column detection, validation, cleaning, feature engineering, category/merchant/time analysis, anomaly detection, forecasting, budget recommendations, financial health score, insight/recommendation generation. No DOM access.
app.js	UI layer only: file upload & drag-drop handling, navigation, rendering every page, and wiring Chart.js. Calls into engine.js for every number it displays.
report.js	Builds the 9-sheet downloadable Excel workbook from the same computed results, using SheetJS.
demo_data.js	The realistic demo dataset embedded as a JS string, so the demo works even opened directly from disk with no server.
index.html / style.css	Page structure and the visual design system (the “ledger” theme).

Processing pipeline

- **Load** — parse the uploaded CSV/Excel file (or the embedded demo string) into row objects.
- **Detect** — match column headers against known aliases (“Date”, “Txn Date”, “Amount”, “Expense”, etc.) to auto-map Date, Amount, Category, Merchant, Payment Method, Type, and Transaction ID.
- **Validate** — check for missing/invalid dates and amounts, duplicates, and dataset size, and surface a clear data-quality report before analysis begins.
- **Clean** — coerce types, standardize category text (whitespace, casing, known synonyms), remove exact duplicate transactions, and log every decision made.
- **Engineer features** — derive year/month/quarter/week/day-of-week, a weekend flag, a transaction-size bucket, and (when income rows exist) monthly income/expense/net-cash-flow/savings-rate.
- **Analyze** — compute category, merchant, time-of-week, and month-over-month trend breakdowns.
- **Detect anomalies** — flag unusually large transactions per category (IQR) cross-checked with a Z-score.
- **Forecast** — blend a moving average with a linear-regression trend on monthly totals.
- **Score** — combine savings rate, spending stability, spending consistency, and discretionary-spend ratio into a transparent, re-weighted 0–100 score.
- **Present** — render the dashboard, tables, and charts, and offer a one-click Excel report export.

05 Data Science Methods Used

Feature	Method	Why this method
Unusual-spending detection	Per-category IQR ($Q3 + 1.5 \times IQR$), cross-checked with a Z-score (threshold 2.5)	Explainable, needs no labeled fraud data, and adapts to each category's own typical spending scale rather than one global threshold.
Spending forecast	3-month moving average blended with a linear-regression trend on monthly totals	Simple and explainable; avoids overfitting a short monthly series with a heavyweight time-series model. Always labeled as an estimate.
Recurring-expense detection	A merchant is flagged recurring if it appears in $\geq 50\%$ of months with amount coefficient-of-variation < 0.25	Captures subscriptions/rent/bills without needing a merchant-category lookup table.
Budget recommendation	Each category's own historical mean and standard deviation, not a fixed percentage of income	Personalized to the individual's actual habits rather than generic advice.
Financial Health Score	Weighted blend of savings rate (35%), spending stability (25%), spending consistency (20%), and discretionary-spend ratio (20%); missing inputs are omitted and remaining weights renormalized	Fully transparent — every sub-score traces back to one specific calculation, and nothing is guessed when data is insufficient.

06 Input — What the User Provides

The application accepts a CSV or Excel (.xlsx/.xls) file of transactions. There is no fixed required schema — the app looks for columns matching common aliases and, if it cannot confidently identify the Date or Amount column, presents a manual mapping panel before continuing. If no file is uploaded, a single click loads a realistic bundled demo dataset instead.

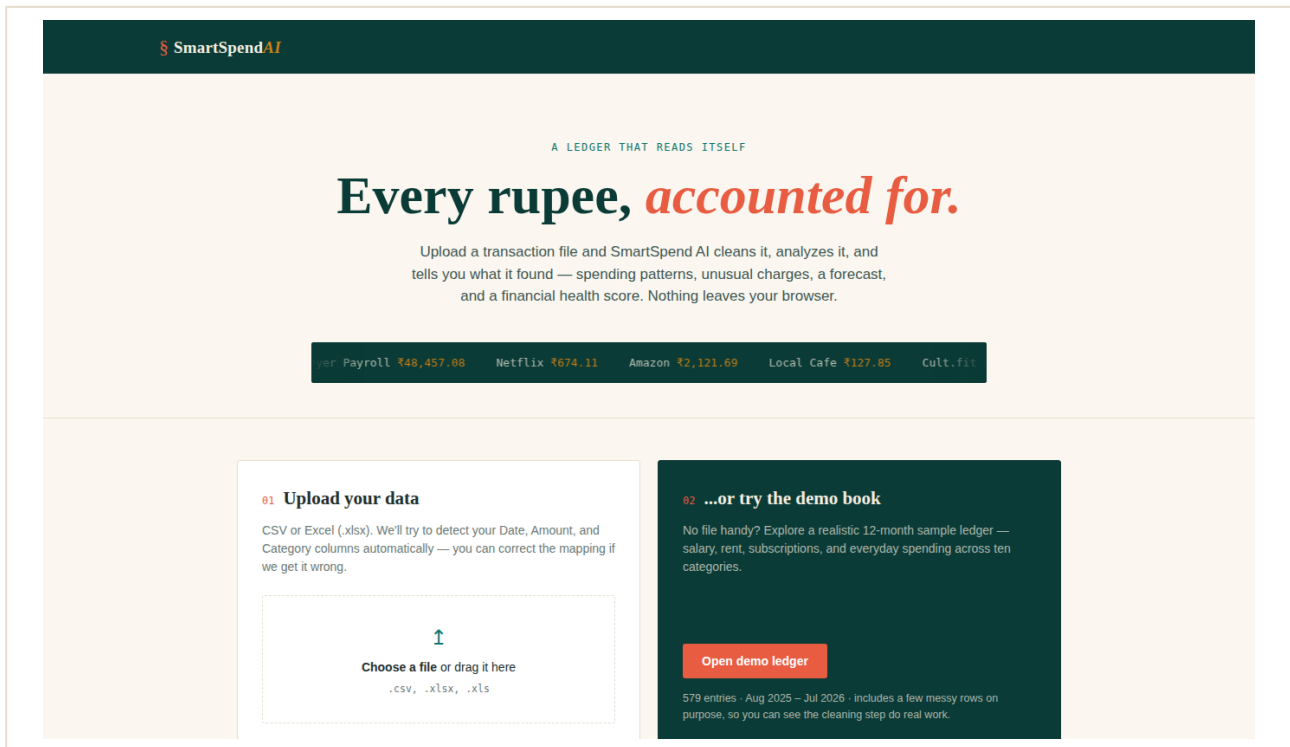


Fig. 1 — Home / intake screen: file upload (drag-and-drop or picker) alongside the one-click demo dataset option.

Demo dataset used for this report

Property	Value
Rows	579 transactions (after cleaning)
Date range	1 Aug 2025 – 27 Jul 2026 (12 months)
Categories	10: Food, Transport, Shopping, Rent & Housing, Entertainment, Health & Fitness, Education, Travel, Personal Care, Bills & Utilities
Includes income rows	Yes — monthly salary plus occasional freelance income
Intentional messiness	A few duplicate rows and inconsistent category casing (e.g. 'food' / 'FOOD'), so the cleaning step has real issues to fix

07 Output — Application Walkthrough

7.1 Dashboard — KPI summary

Once data loads, the Ledger overview page shows headline KPIs computed directly from the cleaned data, followed by seven charts (monthly trend, category split, month-by-month bar, weekday vs. weekend, payment method split, top merchants, and a spending-amount histogram).

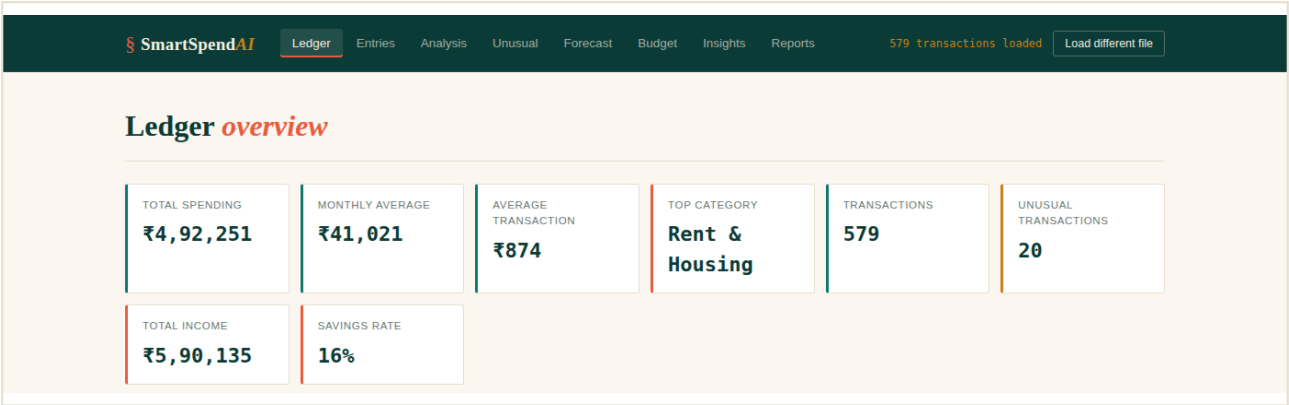


Fig. 2 — Dashboard KPI cards: total spending, monthly average, average transaction, top category, transaction count, unusual-transaction count, total income, and savings rate.

7.2 Transactions — filterable ledger

Every transaction is listed in a searchable, filterable table (date range, category, merchant, payment method, amount range), with unusual transactions flagged inline. The filtered set can be exported as CSV.

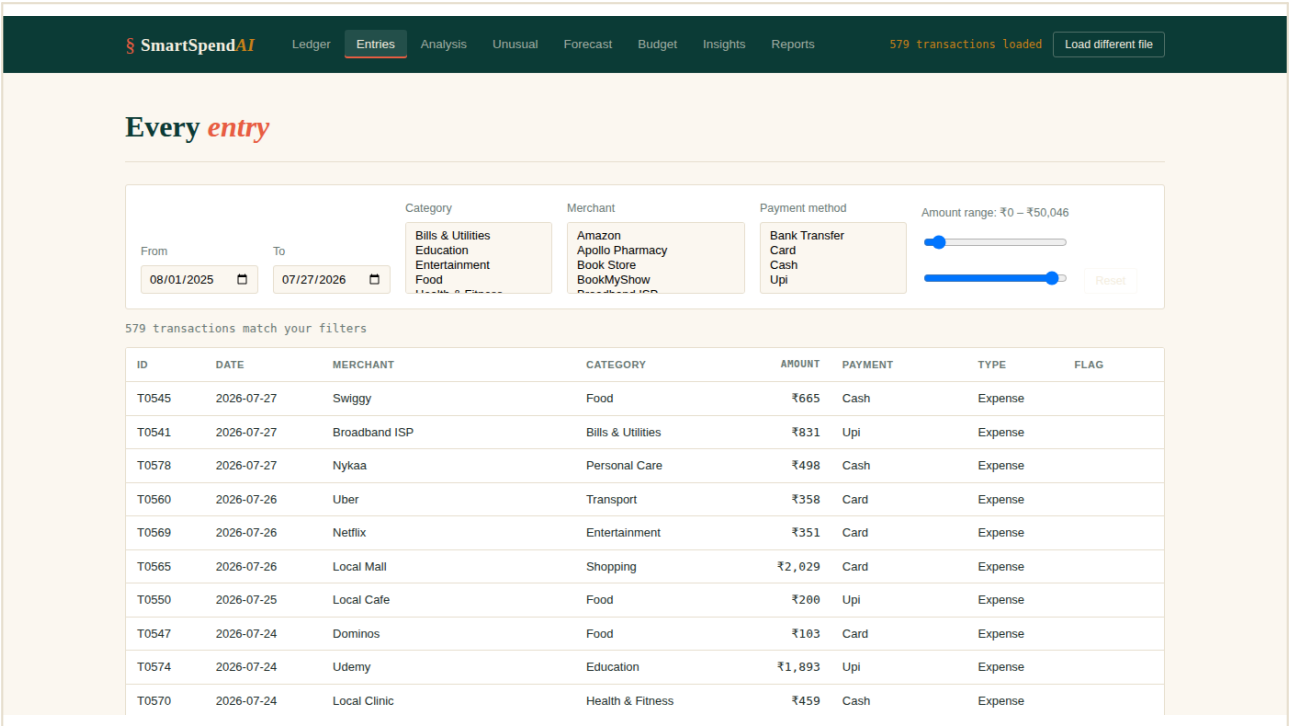


Fig. 3 — Transactions page with active filters and the unusual-transaction flag column.

7.3 Anomaly detection — unusual spending

Transactions are never labeled as fraud — only as “unusual” relative to the person's own spending history, since there is no fraud ground-truth in personal transaction data.

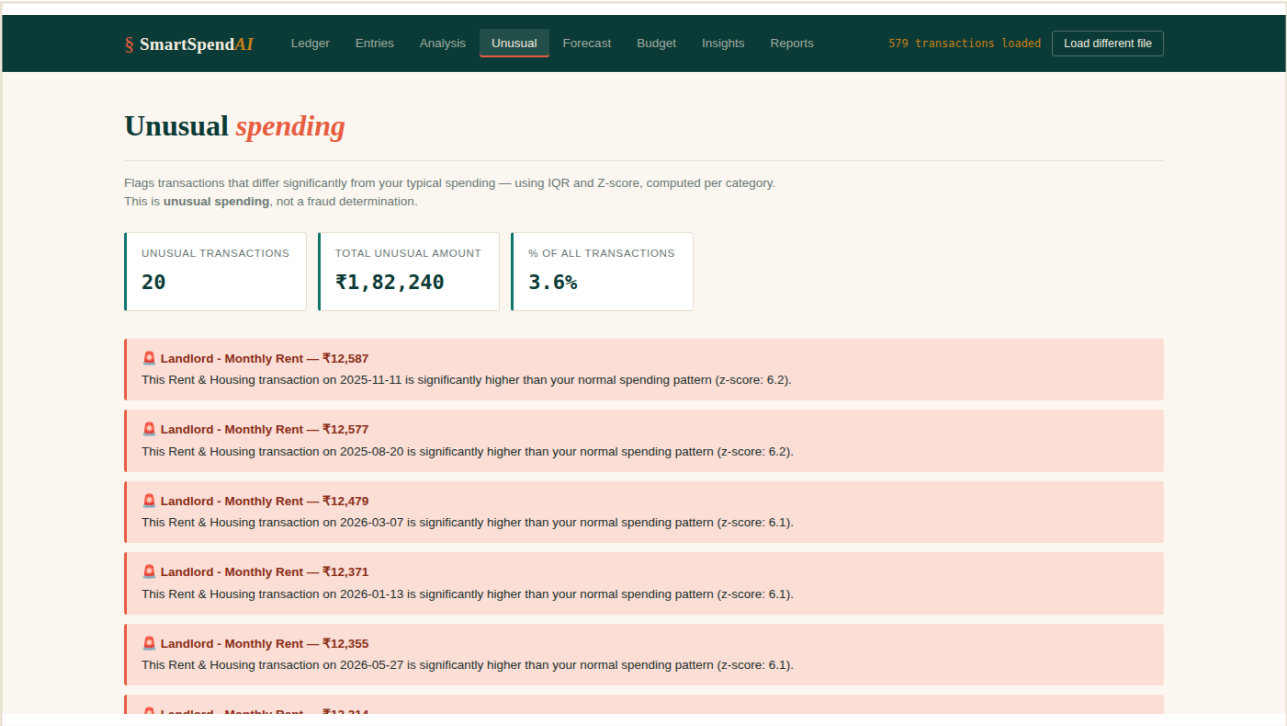


Fig. 4 — Anomaly detection page: KPI summary, individual unusual-transaction cards, and the full unusual-transactions table with Z-scores.

7.4 Budget analysis

Recommended budgets are derived from each category's own historical average, not an arbitrary percentage. Entering a budget produces a live Budget vs. Actual comparison with a status per category.

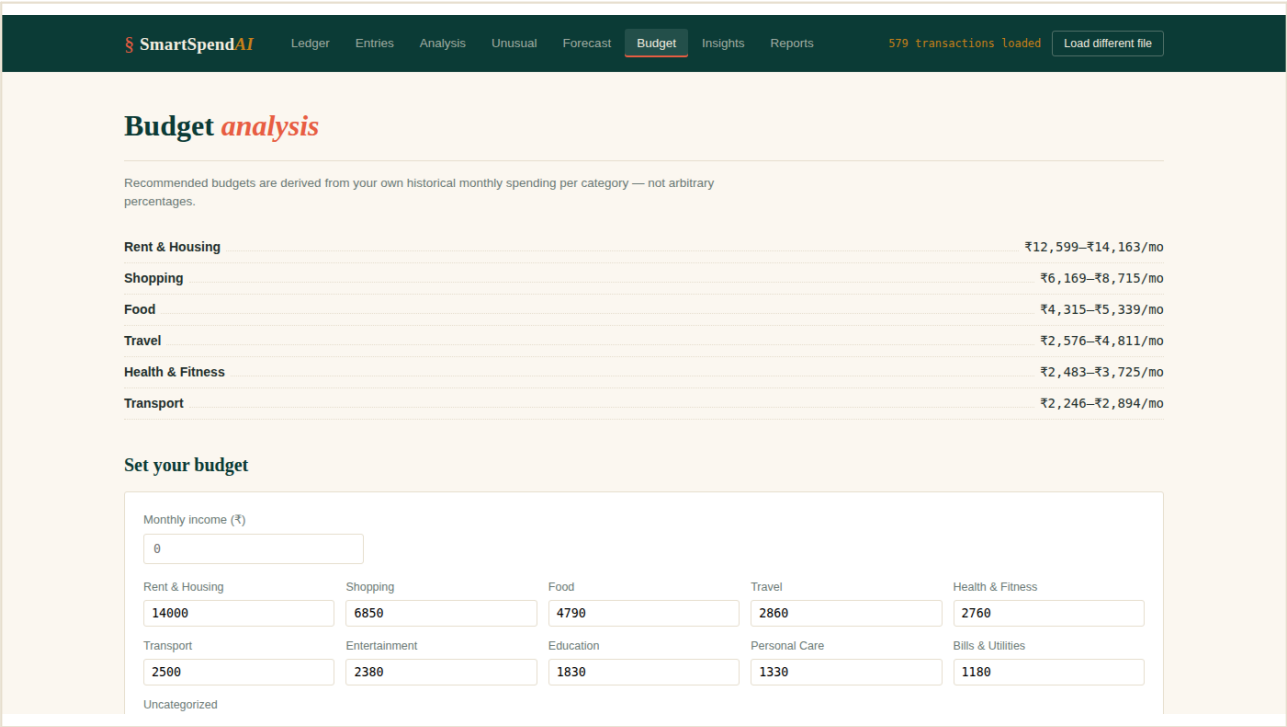


Fig. 5 — Budget analysis: data-driven recommendations at top, then a live Budget vs. Actual comparison with status bars once budgets are entered.

7.5 Smart Insights — the financial health score

The SmartSpend Analytical Score (0–100) is shown with a full breakdown of every sub-score and the calculation behind it, followed by dynamically generated insight and recommendation text — every sentence here is built from a live number, not a hardcoded template.

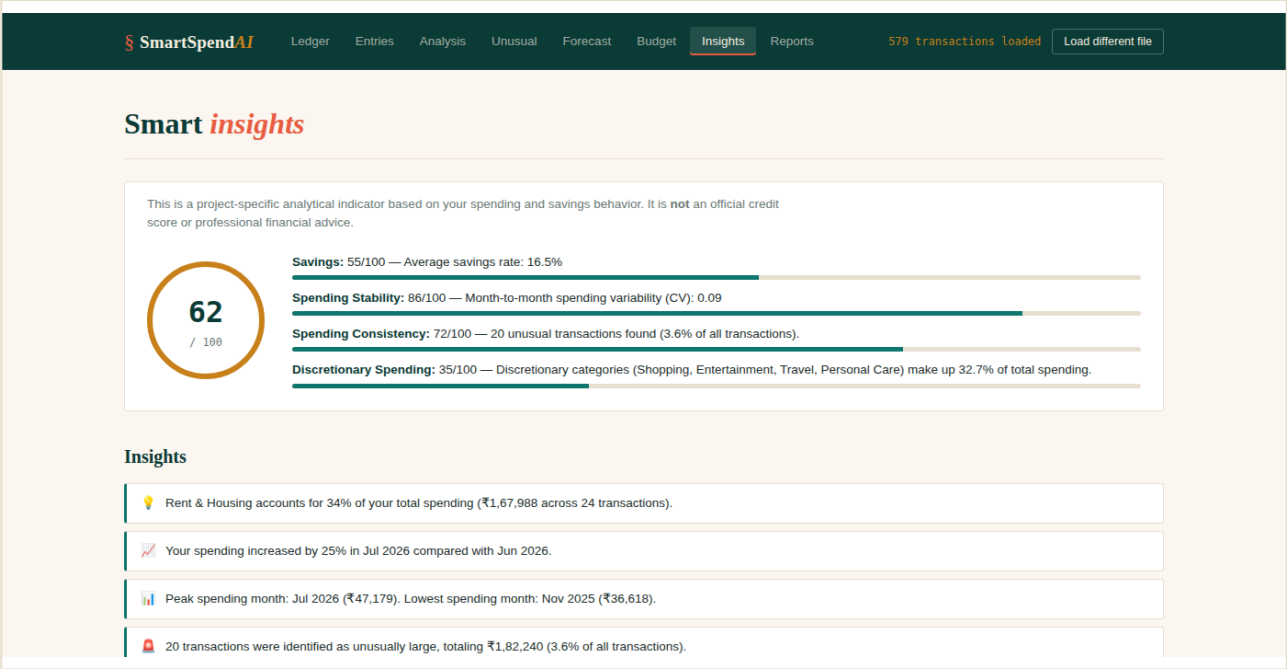


Fig. 6 — Smart Insights page: score breakdown with reasoning, followed by generated insights and recommendations.

7.6 Reports — Excel export

A single click generates and downloads a 9-sheet Excel workbook (Executive Summary, Cleaned Transactions, Spending Analysis, Category Analysis, Anomalies, Forecast, Budget Analysis, Smart Insights, Data Dictionary) built client-side with SheetJS from the same computed results shown on screen.

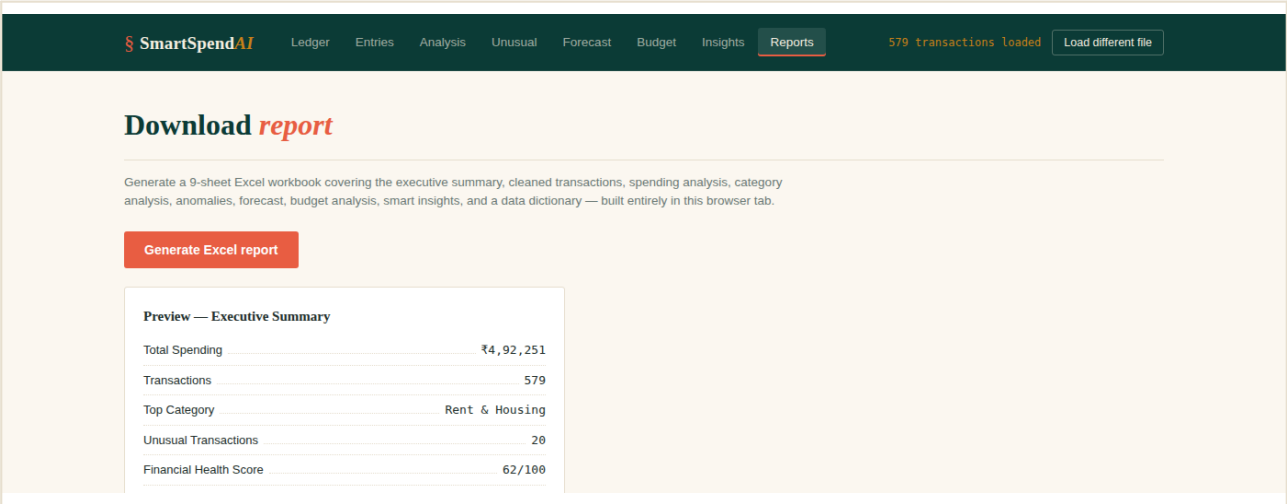


Fig. 7 — Reports page with the executive-summary preview and the one-click Excel export button.

08 Sample Results on the Demo Dataset

The visualizations below are generated from the same computed aggregates as the live dashboard, reproduced here at report resolution.

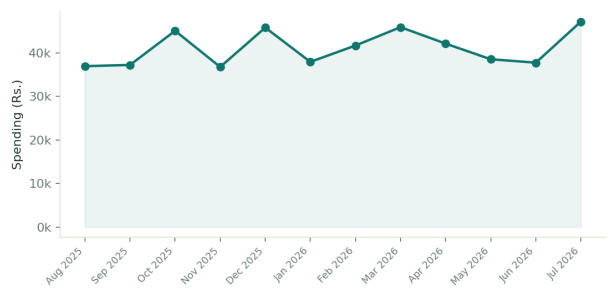


Fig. 8 — Monthly spending trend

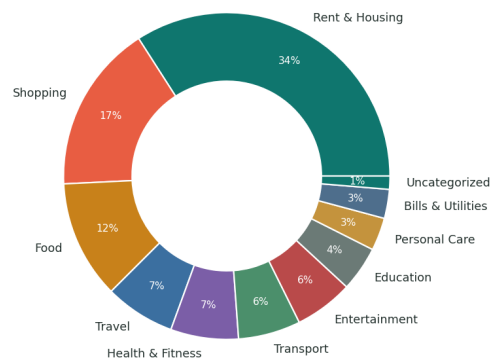


Fig. 9 — Spending by category

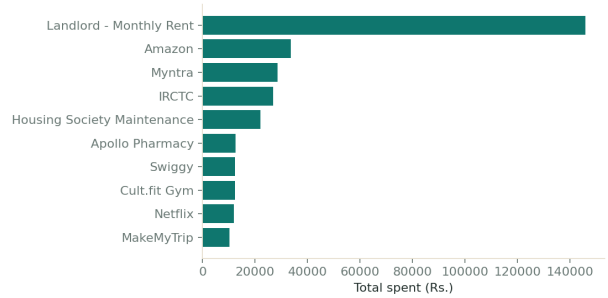


Fig. 10 — Top merchants by spend

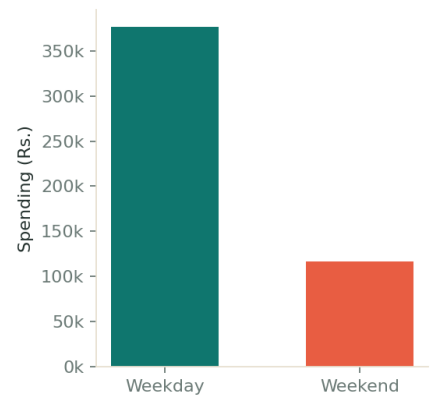


Fig. 11 — Weekday vs. weekend spending

Headline numbers

Metric	Value
Total spending	Rs. 4,92,251
Average transaction	Rs. 874
Top category	Rent & Housing (34% of total spend)
Unusual transactions found	20 (3.6% of all transactions)
Average savings rate	16.5%
Financial Health Score	62 / 100
Largest single anomaly	Rs. 11,645 — Landlord monthly rent (flagged for being above the category's typical range, not because rent itself is unusual)

09 Testing & Verification

The analysis engine and UI were verified in two layers before this report was written:

- **Engine-level unit tests** — every function in engine.js (parsing, cleaning, feature engineering, each analysis method, anomaly detection, forecasting, budgeting, scoring, insight generation) was run directly under Node.js against the real demo CSV and checked against expected values.
- **Full browser end-to-end tests** — the entire UI was driven in a real headless Chromium browser (via Playwright): file upload, drag-and-drop, the column-mapping fallback, every one of the 8 pages, live filtering, budget entry, CSV export, and Excel report generation were all exercised automatically.

This process caught and fixed one real layout bug: on narrow mobile viewports, the top navigation bar could collapse to zero width due to a CSS flexbox / overflow-x:auto interaction. It was fixed by giving the navigation its own full-width row on small screens.

10 Deployment

Because the application has no backend, it deploys as a plain static site — no build step, no server configuration.

Source control

- Repository: github.com/Sadhiya0201/SmartSpendAI
- Workflow: edit locally in VS Code → `git add -A` && `git commit -m "..."` && `git push` via Git Bash.

Hosting

- **Vercel** — the repository is imported directly (Framework Preset: Other, no build command, no output directory needed). Every push to the main branch triggers an automatic redeploy.
- **GitHub Pages** — available as a zero-cost alternative via Settings → Pages → Deploy from branch (main / root).

A one-time nested-folder issue was identified and corrected during setup: the project folder had been unzipped inside a folder of the same name, producing a duplicated path (SmartSpendWeb\SmartSpendWeb\...). Git commands were run from the inner folder so that `index.html` sits at the repository root, which is required for the static host to serve it correctly.

11 Limitations & Future Improvements

Known limitations

- Anomaly detection uses IQR + Z-score only (no Isolation Forest / ML model) — a deliberate trade-off since a Python ML library isn't practical to run client-side in the browser.
- Forecasting is intentionally simple (moving average + linear trend); it will not capture seasonality and is always labeled as an estimate, not a guarantee.
- Recurring-expense detection is a frequency/consistency heuristic, not a merchant-category lookup — it can miss irregularly-billed subscriptions.
- Income-dependent features (savings rate, net cash flow) and multi-month features (forecasting, trends, recurring detection) are explicitly marked *Not Applicable* when the uploaded data doesn't support them, rather than being estimated.

Possible future improvements

- Optional multi-user accounts with persisted history across browser sessions.
- PDF export of the executive summary alongside the existing Excel report.
- Bank-statement-specific parsers for common export formats.
- A seasonal forecasting model once enough historical months are available.

12 Conclusion

SmartSpend AI demonstrates a complete, working personal-finance analytics pipeline — from raw file upload through cleaning, feature engineering, statistical analysis, and a formatted report — built entirely as a static website with no backend. Every number shown to the user is traceable to an explicit calculation in `engine.js`, verified against the demo dataset with both Node-level unit tests and full browser end-to-end tests. The project is deployed from github.com/Sadhiya0201/SmartSpendAI and hosted live on Vercel.

SmartSpend AI is an educational/portfolio analytics project. It is not a licensed financial advisory tool, and the SmartSpend Analytical Score is not an official credit or financial health score.